

Week 2 – Session 2 – Lab 2

Section A: Relational and NoSQL Database Fundamentals

A1. Payment Processing System

A fintech app needs to debit one account and credit another atomically, with strict rollback if either step fails.

Answer- Relational Database like MySQL/PostgreSQL as it is a fintech app and it has atomicity and fixed schema that either it must be credited or it should roll back if not credited.

A2. E-commerce Product Catalog

Products range from books to electronics to furniture — each category has wildly different attributes (e.g., "page count" vs "battery life" vs "dimensions").

Answer – NoSQL documentDB like MongoDB Since it has Flexible schema as it is a product catalog and different product types have different attributes.

A3. Social Media Friend Recommendations

The app needs to suggest "people you may know" by finding friends-of-friends up to 3 levels deep.

Answer – NoSQL - graphDB so Neo4j Since it uses recommendations engines

A4. IoT Sensor Network

Thousands of sensors report temperature readings every second; the system must handle very high write throughput, and reads are mostly "give me the last 24 hours for sensor X."

Answer – NoSQL – Column – Family Databases like Cassandra or HBase as this type of NoSQL model organizes rows by primary key and groups of columns optimized for handling high write throughput data

A5. Session/Cart Management

A retail site needs to store each logged-in user's shopping cart for fast retrieval by session ID, with no complex querying required.

Answer – NoSQL – Key-Value Database like Redis as it stores user info and helps in fast Lookups by key so it will help in fast retrieval using session ID

A6. Financial Reporting Dashboard

A finance team needs to join customer, order, and payment tables to produce monthly revenue reports with strict accuracy.

Answer - Relational Database like MySQL/PostgreSQL as it is customer orders and it has strict accuracy (strict schema's)

Section B: Multiple choice questions

B1. What does a foreign key primarily enforce?

- a) Query performance
- b) Referential integrity between tables
- c) Schema flexibility
- d) Data compression

Answer – b

B2. Which statement about NoSQL is most accurate?

- a) NoSQL means no schema at all
- b) NoSQL databases can never scale
- c) NoSQL favors flexible schema over strict enforcement
- d) NoSQL databases don't support any data validation

Answer- c

B3. In the ACID model, "Isolation" refers to:

- a) Data being stored on isolated servers
- b) Concurrent transactions not interfering with each other
- c) Backups being isolated from production
- d) Tables being isolated from each other

Answer – b

B4. Which NoSQL type is best suited for "friends of friends" traversal queries?

- a) Key-value
- b) Document
- c) Column-family
- d) Graph

Answer – d

B5. BASE stands for:

- a) Basic Availability, Stateless Execution
- b) Basically Available, Soft state, Eventual consistency
- c) Balanced Access, Secure Encryption
- d) Batch Aggregation, Synchronous Execution

Answer – b

B6. A primary key automatically implies:

- a) The column is encrypted
- b) An index is often created for uniqueness enforcement
- c) The table has no foreign keys
- d) The column must be a string type

Answer – b

B7. Which use case is the best fit for a column-family database?

- a) Bank account transfers
- b) Per-user daily time-series metrics with many optional fields
- c) Storing a single user's session token
- d) Modeling friendships between users

Answer – b

B8. What is a key trade-off when choosing BASE over ACID?

- a) You gain strong consistency but lose availability
- b) You gain availability/partition tolerance but accept eventual consistency
- c) You gain faster joins but lose indexing
- d) You gain schema enforcement but lose flexibility

Answer – b

B9. Embedding line items inside an order document (vs. referencing them) is a pattern typical of:

- a) Column-family databases
- b) Graph databases
- c) Document databases
- d) Key-value databases

Answer – c

B10. Which scenario is the weakest fit for a graph database?

- a) Fraud ring detection across connected accounts
- b) Mutual friend discovery
- c) Large-scale tabular financial reporting with millions of rows
- d) Recommendation engines based on relationships

Answer - c

Section 3: Theory Questions

C1: Explain the difference between a primary key and a foreign key, and describe one real-world consequence of missing a foreign key constraint.

Primary Key - Is a column that uniquely identifies each row in a table, It must be unique not null. Each table has one Primary Key

Foreign Key – Is a column in one table that refers the primary key of another table creating a link between two tables.

If we miss a foreign key constraint, we end up with orphan records an order referencing a customer that does not exist.

C2: Compare ACID and BASE across consistency, availability, and typical use cases. Give one example use case for each that was not used in the session notes

Dimensions	ACID	BASE
Consistency	Strong	Eventual
Availability	Low	High
Use case	Airline Seat booking	Gaming Leaderboards

C3: "NoSQL means no schema." Explain why this is a common misconception and describe what actually happens to schema in a NoSQL system.

NoSQL means no schema is a misconception because it will be taken in a literal meaning that it has no schema but actually it is Not only SQL.

Here in NoSQL the schema is flexible schema meaning the database can often accept evolving schemas.

C4: Describe the four main NoSQL types (document, key-value, column-family, graph) in one sentence each, and give one data engineering example per type.

NoSQL documentDB – They store semi structured documents (JSON/BSON) each document may have different fields.

Ex : Storing order events with variable fields, product attributes evolving over time

NoSQL – Key-Value Database - Store of (key -> value) pairs; value can be opaque. Extremely fast lookups by key.

Ex : Redis storing session tokens or user feature toggles.

NoSQL – column-family database – Data model organizes rows by a primary key and groups of columns (column families); optimized for wide, sparse data and high write throughput.

Ex: Storing per-user daily metrics where not all days have values

NoSQL – graph database - Store nodes and edges; optimized for traversals and relationship queries.

Ex: Finding mutual friends or paths between accounts to detect fraud rings.

C5: A team wants to migrate a fixed-schema product table to a flexible-schema document model because product attributes vary heavily by category. What is gained, and what risk should the team watch for after migration?

If they migrate they will have a flexible schema as they can often add new schemas in a database which might have been one of the problem they might have faced. It allows for each product categories to store different attributes.

Once they migrate it becomes a flexible schema and it won't be a strict schema anymore so you may enter wrong data type or insert wrong fields or miss fields so they should enforce data contracts and validation in the ingestion layers.

Section D: Practical Questions

D1: Relational -Build and Query

Using sqlite3, create two tables: employees(employee_id PK, name, dept_id FK) and departments(dept_id PK, dept_name). Insert at least 2 departments and 3 employees. Write a query that joins both tables to list each employee's name alongside their department name.

```
1 CREATE TABLE employees(  
2     employee_id INT PRIMARY KEY,  
3     name VARCHAR(50),  
4     dept_id INT,  
5     FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
6 );  
7 CREATE TABLE departments(  
8     dept_id INT PRIMARY KEY,  
9     dept_name VARCHAR(5)  
10 );  
11  
12 INSERT INTO employees(employee_id,name,dept_id) VALUES  
13 (1,'ALEX',10),  
14 (2,'BOB',20),  
15 (3,'ALICE',30);  
16 INSERT INTO departments(dept_id,dept_name) VALUES  
17 (10,'IT'),  
18 (20,'HR'),  
19 (30,'FINANCE');  
20  
21  
22 SELECT e.name,d.dept_name  
23 FROM employees e  
24 JOIN departments d ON e.dept_id=d.dept_id  
25 GROUP BY d.dept_id;
```


D2: ACID - transaction rollback

Using sqlite3, create a wallet(user, balance) table. Simulate a transfer of 30 units from one user to another inside a transaction. Deliberately raise an exception before committing, then roll back and print the table to confirm balances are unchanged.

```
1 CREATE TABLE wallet(
2   user VARCHAR(50),
3   balance Decimal);
4
5 INSERT INTO wallet(user,balance) VALUES
6   ('Alice',100.50),
7   ('Bobi',200.50);
8
9 BEGIN TRANSACTION;
10
11 UPDATE wallet
12 SET balance = balance - 50
13 WHERE user = 'Alice';
14
15 UPDATE wallet
16 SET balance = balance + 50
17 WHERE user = 'Bobi';
18 -- Simulate failure
19 ROLLBACK;
20 SELECT * FROM wallet;
```

D3: Document Model - Variable Attributes

Using Python dictionaries, create a small in-memory "collection" (list of dicts) of at least 3 products from different categories (e.g., book, laptop, chair), where each product has different fields beyond a shared sku and name. Write a function that filters and returns only products that have a battery_life field.

```
Products = [
    {
        "sku" : "A101",
        "name": "The art of making money",
        "category" : "book",
        "pages" : 350,
        "author" : "unknown"
    },
    {
        "sku" : "B102",
        "name": "ultrabook pro max",
        "category" : "Laptop",
        "RAM" : "16GB",
        "battery_life" : "10 hours"
    },
    {
        "sku" : "C103",
        "name": "IKEA chair",
        "category" : "chair",
        "price" : 3500,
        "capacity" : "100KG"
    }
]
def filter_battery_life(items):
    for item in items:
        if "battery_life" in item:
            print(item)
result=filter_battery_life(Products)
print(result)

{'sku': 'B102', 'name': 'ultrabook pro max', 'category': 'Laptop', 'RAM': '16GB', 'battery_life': '10 hours'}
```

D4: Key-Value Store - Simulated Cache

Using a Python dict as a key-value store, write code to store 3 user sessions keyed by "session:<id>", then write a function get_session(session_id) that returns the session or None if not found (simulating a cache miss).

```
sessions = [
    {
        "user1": "Alice",
        "session_id": "1",
        "session_time": "2hours"
    },
    {
        "user2": "Alex",
        "session_id": "2",
        "session_time": "2hours"
    },
    {
        "user3": "Amit",
        #"session_id": "1", no session id here
        "session_time": "2hours"
    }
]

def get_session(session_list):
    for session in session_list:
        if "session_id" in session:
            print(f"session_id: {session['session_id']}")
        else:
            print("No session_id for this entry")

Result=get_session(sessions)
```

```
session_id: 1
session_id: 2
No session_id for this entry
```

D5: Eventual Consistency

Using two Python dicts (replicaA, replicaB) to simulate replicas, write a value to replicaA only, show that reading it from replicaB returns nothing (stale/missing), then simulate a sync and show replicaB now has the value. Add a short comment explaining which BASE property this demonstrates.

```
replicaA={}
replicaB={}

replicaA["Device"]={"name":"Ultra pro max","category":"laptop"}

print("Read from replicaA: ",replicaA.get("Device"))
print("Read from replicaB: ",replicaB.get("Device"))

replicaB.update(replicaA)

print("After sync , read from replicaB:",replicaB.get("Device"))
```

```
Read from replicaA: {'name': 'Ultra pro max', 'category': 'laptop'}
Read from replicaB: None
After sync , read from replicaB: {'name': 'Ultra pro max', 'category': 'laptop'}
```

Eventually Consistent – BASE property

The system may return stale/missing data temporarily but replica will become consistent eventually after synchronization.

D6: Graph - Friends of Friends

Using an adjacency list (dict of lists) representing a small social graph with at least 4 users, write code to compute the friends-of-friends for one user, excluding the user themselves and their direct friends.

```
graph = {
    "Alice": ["Bob", "Charlie"],
    "Bob": ["Alice", "David"],
    "Charlie": ["Alice", "Eve"],
    "David": ["Bob"],
    "Eve": ["Charlie"]
}

def friends_of_friends(graph, user):
    direct_friends = set(graph.get(user, []))
    fof = set()

    # direct friend's friends
    for friend in direct_friends:
        fof.update(graph.get(friend, []))

    # Remove the user and their direct friends
    fof.discard(user)
    fof = fof - direct_friends

    return fof

result = friends_of_friends(graph, "Alice")
print(result)
```

```
{'Eve', 'David'}
```